

# Day 18 — Production GitHub Actions

## Day 18 — Production GitHub Actions Lab

### Objective

Assemble a full production-style CI/CD pipeline: build, test, Docker build, Docker push, and a protected production deploy.

### Mental Model

Developer  
-> git push  
-> GitHub  
-> GitHub Actions  
-> Build  
-> Test  
-> Docker Build  
-> Docker Tag  
-> Docker Push  
-> Docker Registry  
-> Production Environment  
-> Approval  
-> Deploy

### Tasks

#### 1. Set up registry credentials

Add DOCKERHUB\_USERNAME and DOCKERHUB\_TOKEN as repository secrets.

Never commit real credentials into the workflow file itself.

#### 2. Run the production pipeline

Copy ../../workflows/production-cicd.yml into .github/workflows/.

Push to main and follow the three jobs: build-and-test ->

docker-build-push -> deploy-production.

#### 3. Create a protected environment

In **Settings -> Environments**, create an environment named production, and add a required reviewer. Re-run the workflow and confirm it pauses at deploy-production awaiting approval.

#### 4. Verify image traceability

Confirm the pushed image is tagged with \${{ github.sha }} and that you can match a running container back to the exact commit that built it.

## 5. Simulate common failures

- Break the checkout step (typo the uses: version) -> observe failure.
- Break a test on purpose -> confirm docker-build-push never runs.
- Use a wrong secret name -> confirm the Docker login step fails cleanly.
- Use an invalid Dockerfile path -> confirm the build step fails with a clear error.

## 6. Write the incident notes

For each simulated failure, note: what broke, what the log said, and how you'd fix it. This feeds ../../troubleshooting/.

## Success Criteria

- ☐ Full pipeline runs green end-to-end on a real change
- ☐ Production deploy requires manual approval
- ☐ Every production image can be traced back to a commit SHA
- ☐ You've intentionally broken and diagnosed at least 3 failure types

# Docker-in-CI Issues

## Docker build fails

- **Dockerfile not found:** confirm the build context and `-f` path match  
where the Dockerfile actually lives relative to `$GITHUB_WORKSPACE`.
- **COPY fails / file not found:** build context doesn't include the files  
you're copying — check `.dockerignore` isn't excluding something needed.
- **Base image pull fails:** registry rate limit (common with unauthenticated  
Docker Hub pulls) — log in even for pulling public images to raise limits.

## Docker login fails

- Secret name typo — `${{ secrets.DOCKERHUB_TOKEN }}` must exactly match the  
name configured in **Settings -> Secrets and variables -> Actions**.
- Using a password instead of an access token — most registries require a  
scoped token, not your account password.
- Token expired or revoked — regenerate and update the repository secret.

## Docker push fails

- **"denied: requested access to the resource is denied":** the logged-in  
account doesn't have push rights to that repository/namespace.
- **Tag mismatch:** pushing `myapp:latest` without having built/tagged that  
exact name first — tags must be created locally before `docker push`.
- **Multi-arch build issues:** pushing a `buildx` multi-platform image  
requires `--push` as part of the build command, not a separate push step.

## Good practices baked into the workflows in this module

- Always tag with `${{ github.sha }}` for traceability, in addition to any  
floating tag like `latest`.
- Use `docker/login-action` instead of raw `docker login` with inline  
credentials.
- Scope workflow permissions: to only what's needed (e.g.

packages: write when pushing to GitHub Container Registry).

# Day 18 Interview Questions — Production GitHub Actions

## Q1. Walk through a production CI/CD pipeline end-to-end.

Developer pushes -> GitHub Actions triggers -> build -> test -> Docker

build -> Docker tag (with commit SHA) -> Docker push to registry ->

production environment gate -> manual approval -> deploy.

## Q2. Why tag Docker images with `github.sha` instead of only `latest`?

Traceability — `latest` is a floating pointer, so you can't tell which

commit produced a running container. The SHA tag ties every image to an exact, immutable commit.

## Q3. How do you gate a production deploy behind manual approval?

Declare `environment: production` on the deploy job, then add a required reviewer rule to that environment under repo Settings -> Environments. The job pauses until approved.

## Q4. The checkout step fails — what do you check first?

Whether `actions/checkout` is even present, the `ref` value if one was

supplied, `fetch-depth` if history is needed, and whether submodules need

with: `submodules: true`.

## Q5. Docker push fails with "access denied" — likely causes?

Login succeeded but the account lacks push rights to that

repository/namespace, or the image name/tag doesn't match a repo the credentials can push to.

## Q6. A secret appears to be empty when used — how do you debug it?

Confirm the secret name matches exactly (case-sensitive), confirm it's

defined at the right scope (repo vs environment vs org), and confirm the

job declares the right `environment`: if the secret is environment-scoped.

## Q7. What's the difference between a build failure and a runner failure?

A build failure is your code/tests/Dockerfile — the job started fine, but a

step's command failed. A runner failure means the job never got a machine

to run on (queued forever, label mismatch, self-hosted runner offline).

**\*\*Q8. How would you design branch-based deployment (e.g. `develop` -> staging, `main` -> production)?\*\***

Use `if:` conditions keyed on `github.ref` per deploy job, or use two

separate environments with different `deployment branch policies`, each

job pointing at its own environment.

**Q9. What's a quality gate, and name two examples in a CI/CD pipeline.**

An automated or manual checkpoint a change must pass before proceeding.

Examples: tests must pass before Docker build runs (needs:), and a human must approve before a production deploy proceeds (environment protection rule).

**\*\*Q10. How do you keep production credentials out of the repository entirely?\*\***

Store them as GitHub Secrets (repo/environment/org-scoped) or, better, use OIDC federation to assume a short-lived cloud role at runtime instead of storing any static credential at all.